

# Spring AOP - 1. 왜 AOP 를 쓰는가?

Spring Boot 를 사용하면서 최소 한 번 이상은 Spring AOP 를 들어봤을 것이다.

특히 트랜잭션 어노테이션을 사용하게 된다면 반드시 언급된다.

하지만 왜 AOP 를 쓰는지, 내부에서는 어떻게 동작하는지 등에 대해 나는 알지 못했다. 따라서 공식 문서를 Claude가 번역하여 학습한 내용에 관하여 정리를 해보았다.

번역을 하는 과정에서 틀린 내용이 있을 수 있을 수 있다.

---

## 1.1 문서가 AOP 를 정의하는 방식

문서 첫 문단이 AOP 가 OOP를 대체하는 것이 아닌 보완하는 것이라고 말한다.

**OOP에서 모듈화의 기본 단위는 클래스, AOP는 Aspect 이다.**

Aspect가 모듈화하는 대상은 여러 타입과 객체를 가로질러 걸쳐있는 관심사로 대표적인 것으로 `@Transactional` 이 있다.

이러한 관심사를 AOP 문서에서는 crosscutting 관심사라고 부른다.

**crosscutting** : 중복된 코드가 아닌 클래스 경계와 어긋나는 관심사

---

## 1.2 Spring 이 AOP 를 쓰는 목적

문서에서는 Spring Framework 안에서 AOP의 용도를 두 가지로 말한다.

- 선언적 엔터프라이즈 서비스 제공
- 사용자가 직접 custom aspect 구현해서 OOP를 AOP로 보완

`@Transactional` 은 AOP 의 응용사례가 아닌 Spring이 AOP를 만든 첫 번째 이유다.

### 1.2.1 참고사항

미리 만들어진 선언적 서비스만 사용할거라면 Spring AOP를 직접 만들 필요 없고

공식 문서의 챕터 대부분을 건너 뛰어도 된다.

즉, AOP를 공부하는 이유는 쓰기 위해서가 아닌 이미 쓰고 있는 것들이 **왜 그렇게 동작하는지 알기 위해서**이다. IoC 컨테이너는 AOP에 의존하지 않는다. 원치 않으면 AOP를 안 써도 되고 다만 둘을 합치면 강력한 미들웨어가 된다.

---

## 1.3 용어

| Spring 만의 용어가 아니라 AOP 공통 용어다.

해당 부분에 대한 설명은 따로 다룰 예정이다.

왜냐하면 학습하면서도 이게 뭔지 좀 헷갈리는 것도 있고 바로 와닿지 않은 부분이다.

그리고 1.3 부분에 관한 용어들은 앞으로도 꼭 나오고 이해를 하고 가야한다.

## Aspect

여러 클래스에 공통으로 흩어져 있는 기능을 하나로 묶어 둔 모듈이다.

로깅, 트랜잭션, 권한 검사처럼 여러 곳에서 반복되는 코드를 한곳에 모은 것이라고 보면 된다.

Spring에서는 Aspect를 일반 클래스로 만든다.

- **XML(스키마) 방식** : 평범한 Bean으로 등록한다.
- **@AspectJ 어노테이션 방식** : 평범한 클래스에 @Aspect 만 붙인다.

특별한 클래스를 상속하거나 인터페이스를 구현할 필요가 없다.

## Join Point

부가 기능을 끼워 넣을 수 있는 프로그램 실행 중의 한 지점이다.

일반적으로는 메소드 실행이나 예외 처리 같은 지점을 말한다.

Spring AOP에서는 Join Point가 항상 메소드 실행 시점을 의미한다.

필드 접근이나 생성자 호출 같은 지점은 지원하지 않는다.

## Pointcut

여러 Join Point 중에서 부가 기능을 적용할 곳을 골라내는 조건이다.

Advice는 Pointcut과 연결되고, Pointcut 조건에 맞는 모든 Join Point에서 실행된다.

Spring은 기본적으로 AspectJ의 Pointcut 표현식 문법을 사용한다.

```
// 예시: service 패키지 안의 모든 메소드 실행
@Pointcut("execution(* com.example.service.*.*(..))")
```

| 용어         | 의미             | 비유        |
|------------|----------------|-----------|
| Join Point | 적용할 수 있는 후보 지점 | 반 학생 전체   |
| Pointcut   | 그중에서 고르는 조건    | "안경 쓴 학생" |

공식 문서는 이 관계를 **predicate(판정 조건)** 라는 단어로 설명한다.  
Pointcut은 지점을 하나하나 나열한 목록이 아니다. 어떤 지점이 들어오면 적용 대상인지 아닌지를 판단하는 **규칙**이다.

## Advice

선택된 **Join Point**에서 **실제로 실행되는** 부가 기능 코드이다.  
언제 실행되느냐에 따라 다음과 같이 나뉜다.

| 종류              | 실행 시점                                 |
|-----------------|---------------------------------------|
| @Before         | 메소드 실행 전                              |
| @AfterReturning | 메소드가 <b>정상적으로</b> 끝난 후                |
| @AfterThrowing  | 메소드에서 <b>예외가 발생한</b> 후                |
| @After          | 정상 종료나 예외 여부와 관계없이 끝난 후 (finally와 비슷) |
| @Around         | 메소드 실행 전후를 모두 감싸서 제어 (가장 강력함)         |

### 구현 힌트

Spring을 비롯한 많은 AOP 프레임워크는 Advice를 **인터셉터 (Interceptor)** 형태로 구현한다.

그리고 Join Point 주위에 **인터셉터 체인**을 걸어 둔다.

메소드가 호출되면 요청이 이 인터셉터들을 순서대로 거친 뒤 실제 메소드에 도달한다.

서블릿 필터 체인과 비슷한 구조라고 생각하면 된다.

## Introduction

기존 클래스에 새로운 메소드나 필드를 추가로 선언하는 기능이다.

Spring AOP에서는 대상 객체의 코드를 수정하지 않고 **새 인터페이스를 구현한 것처럼** 만들 수 있다.

공식 문서는 캐싱을 쉽게 하려고 Bean이 `IsModified` 인터페이스를 구현하도록 만드는 예시를 든다.

원래 클래스에는 `IsModified`가 없지만, AOP가 "변경되었는지" 확인하는 기능을 외부에서 붙여 주는 것이다.

AspectJ에서는 이 기능을 **inter-type declaration**이라고 부른다.

## Target Object : 타깃 객체

하나 이상의 Aspect에 의해 어드바이스되는 객체이다.

advised object 라고도 한다. Spring AOP 는 Runtime Proxy 로 구현되기 때문에 타깃 객체는 항상 프록시된 객체다.

## AOP Proxy

aspect 계약을 이행하려고 AOP 프레임워크가 만들어낸 객체다.

Spring에서는 **JDK Dynamic** 또는 **CGLIB Proxy** 를 사용한다.

## Weaving

Aspect 를 다른 타입 · 객체와 연결해서 어드바이스된 객체를 만들어내는 작업이다.

시점은 컴파일 타임 · 로드타임 · 런타임 셋 중 하나이다.

Spring AOP 는 다른 순수 Java AOP 프레임워크들과 마찬가지로 런타임에 위빙을 수행한다.

### 1.4 Advice 의 종류

- `Before` : `Join point` 이전에 실행하며 실행 흐름을 막을 능력이 없다. 단, 예외를 던지면 얘기가 달라진다.
- `After returning` : `Join point` 가 정상 완료된 뒤 실행하며 예외 없이 반환한다.
- `After throwing` : 메소드가 예외를 던지며 빠져나갈 때 실행한다.

- `After(finally)` : 정상이든 예외든 어떤 방식을 빠져나가든 실행한다.
- `Around`
  - : `Join point` 를 감싼다. 전 · 후 동작 모두 가능하며 `Join point` 로 진행할지 말지 를 스스로 결정한다.
  - : 자기 반환 값을 돌려주거나 예외를 던져 타깃 실행을 건너뛸 수 있다.

문서가 명시적으로 권고하는 지점이 있다.

필요한 동작을 구현할 수 있는 **가장 덜 강력한 어드바이스 타입을 쓰라고 한다.**

반환 값으로 캐시를 갱신하는 정도라면 `around` 보다 `after` `returning` 이 낫고

더 구체적인 타입을 쓰면 프로그래밍 모델이 단순해지고 실수 여지가 줄어든다.

구체적으로는 `proceed()` 를 호출할 필요 자체가 없으니 호출을 빼먹을 수도 없다.

실무에서는 `@Around` 하나로 다 하는 습관이 흔한데 문서 기준으로는 그게 기본 값이 아니라고 한다. `@AfterThrowing` 이냐 `@Around` 안 `try/catch` 이냐의 판단기준이다.

추가로 어드바이스 파라미터는 정적 타입이라 `Object[]` 를 다루는게 아니라 제대로 된 타입을 받는다.

## 1.5 마무리

Pointcut에 의해 매칭되는 Join point 라는 개념이 AOP의 핵심이며 객체 지향 계층 구조와 독립적으로 타겟팅 되게 해준다.

상속은 객체지향 계층 구조에 종속된 방식이다.

즉, 부모를 공유하는 것들에만 적용이 된다. 반면에 pointcut 은 계층 구조와 무관하게 대상을 고른다.

..service.. 인것

특정 어노테이션이 붙은 것

메소드 이름이 save 로 시작하는 것

--> 상속 계층이 제각각이어도 상관 없다.

유틸 클래스는 코드를 모아줄 뿐 적용 대상을 선언할 수단이 없다. AOP의 차별점은 재사용이 아니라 포인트 컷이다.

| 객체지향과 독립적으로 적용 지점을 지정하는 능력이다.